

1.数据库层 Db 类

Db 类代表一个数据库实例，并管理其中的表。

Db::create_table 方法是处理建表请求的一个关键入口点，它会检查表是否存在，分配表 ID，并调用 Table 对象的创建逻辑。

```
/**
 * @brief 一个 DB 实例负责管理一批表
 * @details 当前 DB 的存储模式很简单，一个 DB 对应一个目录，所有的表和数据都放置在这个目录下。
 */
class Db
{
    /**
     * @brief 创建一个表
     * @param table_name 表名
     * @param attributes 表的属性
     * @param storage_format 表的存储格式
     */
    RC create_table(const char *table_name, span<const AttrInfoSqlNode> attributes, const vector<string> &primary_keys,
        const StorageFormat storage_format = StorageFormat::ROW_FORMAT);
    ...
};
```

2.表对象与元数据处理 (Table 和 TableMeta 类)

Table 类代表一个具体的表。

Table::create 方法是实际执行表创建、元数据初始化和存储文件准备的核心，此方法内部会调用 TableMeta::init 来根据 AttrInfoSqlNode 列表初始化表元数据 TableMeta，并为每一列创建 FieldMeta 对象。

```
/**
 * 创建一个表
 * @param path 元数据保存的文件(完整路径)
 * @param name 表名
 * @param base_dir 表数据存放的路径
 * @param attribute_count 字段个数
 * @param attributes 字段
 */
RC create(Db *db, int32_t table_id, const char *path, const char *name, const char *base_dir,
    span<const AttrInfoSqlNode> attributes, const vector<string> &primary_keys, StorageFormat storage_format,
    StorageEngine storage_engine);
```

FieldMeta 存储了列的详细属性，如类型、长度、名称以及在物理记录中的偏移量。

```
/**
 * @brief 表元数据
 */
class TableMeta
```

3. SQL 解析阶段

当用户输入 `create table` 语句时，MiniOB 首先通过词法分析器 (lex) 和语法分析器 (yacc) 进行处理。

语法规则定义在 `yacc_sql` 中。

`create_table_stmt` 规则用于解析整个建表语句

而 `attr_def_list` 和 `attr_def` 用于解析列定义

解析的结果存储在 `CreateTableSqlNode` 和 `AttrInfoSqlNode` 结构体中

```
// @brief 描述一个属性
struct AttrInfoSqlNode
{...};

// @brief 描述一个 create table 语句
struct CreateTableSqlNode
{...}
```

4. 语句对象构建阶段

`CreateTableStmt` 类持有建表所需的信息

```
// @brief 表示创建表的语句
class CreateTableStmt : public Stmt {...};
```

解析得到的 `CreateTableSqlNode` 随后被 `CreateTableStmt::create` 转换为一个语句对象

5. 执行与调度

`create table` 的 SQL 语句首先被解析成通用的 `ParsedSqlNode`，然后在 `Stmt::create_stmt` 中解析成具体的 `Table` 对象

```
/**
 * @brief Stmt for Statement
 * @ingroup Statement
 * @details SQL 解析后的语句，再进一步解析成 Stmt，使用内部的数据结构来表示。
 * 比如 table_name，解析成具体的 Table 对象，attr/field name 解析成 Field 对象。
 */
class Stmt
{
    static RC create_stmt(Db *db, ParsedSqlNode &sql_node, Stmt *&stmt)
    {
        case SCF_CREATE_TABLE: {
            return CreateTableStmt::create(db, sql_node.create_table, stmt);
        }
    }
    ...
};
```